Appendix D

Benchmarking Structured-Output Scorers

After reading this appendix, you will be able to:

- Identify where typed decisions from language models enter a search and recommendation stack, and which property of an answer (its label, its probability, a threshold on that probability, or its latency) each consumer relies on.
- Distinguish three routes to a structured answer (free text parsed afterwards, constrained decoding, and typed decision heads) and state what each route guarantees and what it leaves open.
- Build a benchmark whose every score can be traced to versioned items, labels, and predictions, and recomputed on a CPU without the models.
- Treat reference labels written by language-model judges as evidence with known biases, and measure how much a comparison of scorers depends on the label source.
- Compute calibration from the probability a decision actually uses, and establish what a field named `confidence` means before binning it.
- Read a results table through its baselines, paired intervals, and stated conditions, and decide which conclusions it can and cannot support.

Search and recommendation teams increasingly ask language models short, closed questions about their data. Is this cable compatible with that laptop? Does this query name a brand? The answers become training and evaluation labels, ranking features, and gates that decide whether a system acts on a decision. Researchers at Bing found that language-model relevance labels agreed with careful searcher feedback at least as well as human labellers did, and that label quality moved unpredictably when the prompt was merely paraphrased, which is why they kept a set of gold labels for choosing and auditing prompts (Thomas et al., 2024).

A newer class of model narrows the question further. A typed decision model reads the input once and returns, for each question, one option from a closed set, a probability for every option, and a summary number usually called `confidence`. Parsing failures disappear by construction, which makes a leaderboard of accuracy and calibration tempting. This appendix examines what an honest comparison needs, through one small benchmark: four commerce tasks, five scorers and a majority-class baseline, labels from two language-model judges and from human annotators, and a replay pack that recomputes every reported number on a CPU. Its results are dated; the methods for building, labelling, scoring, and reading it are not. Every benchmark number quoted here is read from a file that the companion code generates, so each one can be checked.

D.1 Where Structured Outputs Enter a Recommender Stack

A structured output is an answer whose form is fixed before the question is asked: named fields, each with a type and, for the decisions this appendix studies, a closed set of allowed values. In a

recommender, such answers arrive at two places, and each consumer relies on a different property of the answer.

The first place is offline. A batch job asks a model about catalog items, queries, or query-item pairs, and stores the answers. Some become training or evaluation labels, such as a relevance grade for a query and a product, the pattern Chapter 12 (Section 12.4) describes. Others enrich the catalog, for example by recording whether an accessory fits a particular device, so that a complementary-recommendation module can later assemble an "often bought with" slate. For these consumers the label matters most: a wrong compatibility grade becomes a wrong recommendation every time the record is read.

The second place is online. When a shopper submits a query, a query-understanding step may decide what the shopper is trying to do, whether the query names a brand, and which category it targets, before retrieval runs. A retrieval filter consumes the label. A ranking feature usually consumes a probability, and the ranker learns from training data how much to trust a value of 0.7, which holds only while the meaning of 0.7 stays fixed across items and model versions. Online steps also spend part of the request's latency budget (Chapter 14, Section 14.2).

A third consumer, the gate, compares the probability attached to a decision with a threshold. Above it the system acts; below it the system asks the user, routes the item to a person, or sends it to a larger model, as in the cascades of Chapter 12 (Section 12.4.4). A gate depends on something neither accuracy nor a single calibration number reports directly: the error rate among the items that clear the threshold, and the share that clear it.

Figure D.1 places these consumers in one stack and pins the benchmark's four tasks to the entry points they exercise. Query segmentation, which labels a query's goal, target object, specificity, and commerce scope, and brand and category selection, which picks a brand and a category from short candidate lists, stand in for online query understanding. Compatibility, which grades whether a candidate product works with an anchor product, stands in for catalog enrichment. The ESCI task, which grades a query-product pair as Exact, Substitute, Complement, or Irrelevant, stands in for relevance labels.

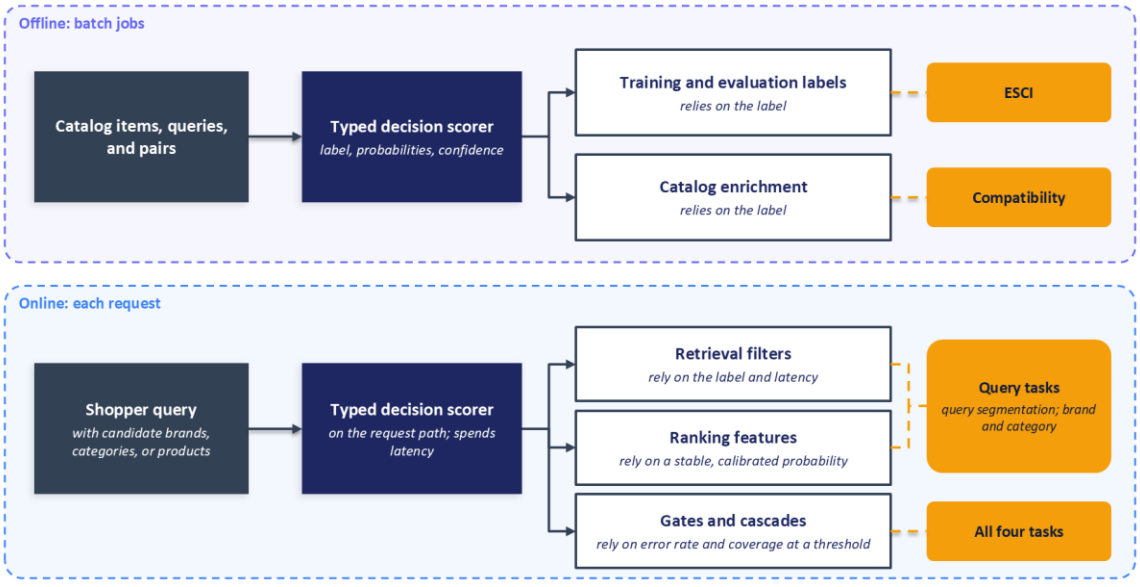


Figure D.1: Structured-output scorers enter a search and recommendation stack offline, as labels and catalog features, and online, as query-understanding decisions and gates. Each consumer relies on a different property of the answer. Query segmentation and brand and category selection feed both retrieval filters and ranking features, and all four tasks can also feed a gate.

Because consumers depend on different properties, no single benchmark number serves them all. Accuracy speaks to labels, calibration and proper scoring rules to features, risk at a given coverage to gates, and serving measurements on equal hardware to latency.

D.2 Three Routes to a Structured Answer

A team that wants a closed answer from a language model can take one of three routes, summarised in Table D.1. They differ in what they guarantee about the form of an answer and in what they reveal about the model's uncertainty.

The first route asks a generative model to write its answer, often as JSON, and parses the text afterwards. The model can return text the parser cannot read, a label outside the allowed set, or a near miss such as "compatible (generic)" that a lenient parser maps onto a valid label, quietly turning an error into an answer; Chapter 12 (Section 12.5.5) describes the validation such outputs need. The text carries no probability over the options, so a team that wants one must ask for a number, which is another generated string.

The second route constrains decoding: at each step, the decoder masks every token that would break a grammar or schema, so the output always parses. Willard and Louf (2023) made this efficient by compiling a regular expression or JSON schema into a finite-state machine indexed against the model's vocabulary, and Chapter 12 (Section 12.2.6) uses the same idea to keep generated item identifiers inside a catalog. Constraint guarantees form, not content, and restricting format can itself change answers: Tam et al. (2024) measured lower reasoning performance under strict format restrictions. The probabilities on this route belong to tokens, so a label spanning several tokens needs its token probabilities combined before they say anything about the option.

The third route skips generation. A typed decision model receives the input once, with questions whose answers come from closed sets, and scores every option of every question in a single forward pass. It returns the selected option, a probability for each option, and usually a summary statistic called `confidence`. One hosted API documents question types that include a choice among named options, a score over ordered levels, and a yes-or-no probability (TypeSafe, 2026b). The answer cannot fall outside the offered options, and every answer arrives with a full probability vector. The failures left on this route are the ones the rest of this appendix studies: a valid answer that is wrong, a probability that means something other than its consumer assumes, and answers that move when the options are reordered.

Table D.1: Three routes to a structured answer.

Property	Free text, parsed afterwards	Constrained decoding	Typed decision heads
What the model produces	Text that a parser maps to fields and labels	Tokens limited at each step to those a grammar allows	A probability for every option of every question
Form of the answer	Can fail to parse	Always parses	Always one of the offered options
Answers outside the option set	Possible unless the parser rejects them	Prevented if the grammar lists the options	Prevented by construction

Property	Free text, parsed afterwards	Constrained decoding	Typed decision heads
Uncertainty exposed	A generated number, if requested	Token probabilities, if the service returns them	A probability per option, plus a summary statistic
Main failure left open	Parse failures and near misses mapped to valid labels	Valid but wrong answers; format pressure on reasoning	Valid but wrong answers; probability meaning; option order

The benchmark compares five scorers on the third route, plus a majority-class baseline. Table D.2 lists them. The prose of this appendix refers to each scorer by its role, because the products change faster than the methods: the hosted API, for example, released a newer version after these runs. The table is the only place that names the products, and its product facts are dated.

Table D.2: The scorers in the September 2026 runs. Product facts are as of 2026-11-09.

Role label	Product and version scored	What it is	Where it ran (as recorded)	Observed meaning of `confidence`
Majority prior	None	One fixed class per field, with probability one	CPU	None
Encoder decision model	Laya, `convaiinnovations/laya`, `typed-decisions` checkpoint	421-million-parameter encoder fine-tuned on four non-commerce workflows; 1,024-token context; Apache 2.0	Local GPU, 8,188 MB, type not recorded	One minus normalised entropy
Hosted typed-decision API	TypeSafe Jev, `jev-1.13.0`	Closed hosted model; `jev-1.14.0` available from October 14, 2026	Hosted API	Mostly the rescaled margin
Open adapter, small	Kev, `jaredpalmer/kev-0.8b`	Adapter on Qwen3.5-0.8B-Base; revision not recorded	Local GPU, 8,188 MB, type not recorded	Rescaled margin
Open adapter, mid	Kev, `jaredpalmer/kev-4b`	Adapter on Qwen3.5-4B-Base; revision not recorded	Colab GPU, 40,960 MB, type not recorded	Rescaled margin
Option-letter adapter	JevLite, `vagmi/jev-lite`	Adapter on Gemma 4 E4B that reads the logits of option letters;	Colab GPU, 40,960 MB, type not recorded	Top-label probability

Role label	Product and version scored	What it is	Where it ran (as recorded)	Observed meaning of `confidence`
		serving precision not recorded		

Two rows need their conditions stated next to them. The encoder decision model is a specialist checkpoint, fine-tuned on four workflows outside commerce (invoice processing, security incidents, customer service, and agent-trace observability), and its card describes the family as a fast base to specialise rather than a zero-shot decision engine and advises refitting temperatures on one's own data before trusting its probabilities (Convai Innovations, 2026). The benchmark also truncated its context. The encoder decision model ran with the adapter's `max_len=512`, although the checkpoint accepts 1,024 tokens. A tokenizer check on 2026-11-09 found inputs cut at 512 tokens for 1,996 of the 2,000 brand and category items and 3 of the 2,000 compatibility items; no input exceeded 1,024 tokens. The checkpoint's 256-token question limit kept 127 to 215 of the 452 instruction tokens in each query-segmentation question. ESCI inputs were not cut. Its results therefore measure an out-of-domain specialist in this configuration, not the checkpoint's best case. The option-letter adapter's card reports that serving precision changes its calibration (Vagmi, 2026), and the committed runs did not record the precision used.

D.3 Building a Benchmark You Can Trace

A benchmark earns trust when any number in it can be traced to the items, labels, and predictions behind it, and recomputed by someone who doubts it. This section builds the benchmark that way, then lists in Table D.3 the failures the construction guards against and those it leaves open.

D.3.1 Tasks, Fields, and Label Spaces

Each task asks one or more closed questions, called fields, about an item. The compatibility task shows a scorer a shopper's query context, an anchor product, and a candidate product, and asks for one of four grades: incompatible, insufficient evidence, generic compatible, or explicit fit. Its 2,000 items come from five sampling strata: candidates generated as cross-sell or direct accessories for the anchor, products that human annotators marked as irrelevant or as substitutes for the query (hard negatives, since a substitute is rarely an accessory), and products drawn from other categories.

The query-segmentation task asks four questions about a query alone: the shopper's goal (six options, from known-item navigation to service and account requests), the kind of object sought, how specific the query is, and its commerce scope. Half of its 2,000 queries come from the Shopping Queries Dataset (Reddy et al., 2022), sampled across six strata of product-type intent. The other half come from ORCAS-I, a web-search query set with gold intent annotations (Alexander et al., 2022), whose non-commercial licence keeps those queries and their intent labels out of the released files; all 1,000 carry a single stratum, `orcas_i`, and the appendix does not analyse them by ORCAS-I intent category.

The brand and category task shows a query with up to eight candidate brands and up to eight candidate category paths, each list ending in an escape option ("none or other brand", "other or ambiguous"). It asks whether the query expresses brand intent, which brand and which category the shopper wants,

and, as a yes-or-no question, whether the query targets several products. Its 2,000 items come from three strata, according to whether the query names a brand, names only a category, or is ambiguous.

The ESCI task grades a query-product pair as Exact, Substitute, Complement, or Irrelevant, using 8,000 pairs from the US test split of the Shopping Queries Dataset, balanced at 2,000 per grade. It is the only task with human labels. Appendix B uses the same dataset to evaluate hybrid search.

D.3.2 Sampling, Pilots, and What Models See

Every sample is drawn with a fixed seed, and the brand and category candidate lists are shuffled with a seeded generator so that the matched candidate lands in different positions across items. A rebuild therefore produces the same items with the same option order. The sampling strata come partly from an earlier language-model enrichment of the ESCI queries, run with GPT-5.1, which supplied a query taxonomy, complement candidates, and category matches. The category paths it matched are the US category paths from ESCI-S, the product metadata named in the source box of Figure D.2, and those paths were shown to the scorers and the judges. None of the enrichment's other text was shown to them.

The rubrics that tell judges how to label were developed on separate pilot sets, labelled on 2026-09-26, one day before the reference labels. Pilot items are excluded from the reference sets: by query text for query segmentation, by query for brand and category, and, for compatibility, by the exact combination of query, anchor, and candidate. Under that last rule, a reference item can share its query and anchor product with a pilot item while pairing them with a different candidate. The replay pack's manifest records 536 of the 2,000 compatibility reference items as sharing a pilot item's query and anchor product, and no query-segmentation or brand and category reference item as sharing a pilot item's query.

A scorer sees a compact state built from an allow-list: the query and, for each product, its title, brand, colour, and one bullet point, clipped to 240, 80, 80, and 180 characters. Item and product identifiers are removed from that state, and two tests in the companion code enforce the allow-list. A judge sees the adjudication input. That input includes the item identifier the judge must return and, on the product tasks, each product's identifier, and it clips the title to 500 characters, the brand and colour to 120, and the bullet point to 700. The judge prompt is built only from that input, so sampling metadata, stratum names, and human labels stay out of it, and a separate test checks that exclusion. An allow-list matters here because a single leaked field, such as a stratum named after the intended answer, would turn the benchmark into a reading test.

D.3.3 Reference Labels and the Provenance Chain

Two judges labelled every item of the three non-ESCI tasks under versioned rubrics whose hashes the pack records. A field receives a reference label only when both judges returned the same value; otherwise its reference is withheld and the field is not scored for that item. Section D.4 examines what this rule does. The majority prior predicts one fixed class per field with probability one, so every table has a row that knows nothing about the input.

Every prediction is stored per item with its full probability vector, and a replay pack keeps identifiers, labels, predictions, and hashes but no text, so the scores can be recomputed without the models or the licensed data (Section D.8). Figure D.2 traces this chain.



Figure D.2: The provenance chain behind every score in this appendix. Counts are read from the replay pack's manifest and label-provenance files. Text stops at the replay pack, which holds identifiers, labels, probabilities, and hashes only.

D.3.4 A Catalog of Failure Modes

The construction above defends against some failures and leaves others open. Table D.3 lists the fifteen that the rest of this appendix examines, with the status of each in this benchmark and the section that discusses it.

Table D.3: Failure modes in structured-output benchmarks, their status in this benchmark, and the guard for each.

Failure mode	What goes wrong	Status here	Guard	Section
Invalid or out-of-set answers	Free text yields labels outside the schema or malformed output	Guarded: typed heads answer within the offered options	A closed answer space; count invalid answers for any other route	D.2
Parser leniency	A normaliser maps near-miss strings onto valid labels	Present on the judge path: display names and aliases map to option identifiers	Log every normalisation with its input and output	D.2

Failure mode	What goes wrong	Status here	Guard	Section
Default threshold on yes-or-no answers	A 0.5 cut assumes informative, calibrated scores	Observed on the multi-target field	Choose thresholds on held-out labelled data	D.5
Leakage into the input	Sampling context reveals the answer	Guarded by an allow-list and tests	Allow-list, not block-list, with a test of allowed keys	D.3
Agreement filter selects easier items	Keeping agreed items drops ambiguous cases	Observed, strongest in query segmentation	Report withheld shares by stratum; score against each judge	D.4
Judge circularity	Judges and scorers share lineage and priors	Open for three tasks	Keep a human-labelled anchor	D.4
Degenerate field	Almost no positives, so accuracy measures the prior	Observed on the multi-target field	Minimum class support; report apart	D.4
`confidence` read as a probability	A summary statistic is binned as if it were a probability	Guarded: calibration computed from probabilities	Name the quantity in every table header	D.5
Option-order bias	Answers depend on the position of options	Shuffled per item; visible by position	Measure accuracy by position; permute	D.6
Wording sensitivity	Meaning-preserving edits move answers	Open: one prompt version	Score several reviewed paraphrases	D.6
Serving numbers from unequal conditions	Hardware, batch size, and network differ	Observed	Same hardware and harness, or no comparison	D.7
Point estimates across many cells	Unpaired numbers invite winner reading	Guarded: paired intervals	Paired bootstrap; name the	D.7

Failure mode	What goes wrong	Status here	Guard	Section
			comparisons in advance	
Cache keyed on a hand-set version	Edited prompts return stale predictions	Open	Key the cache on a hash of the rendered prompt	D.8
Interpretation written by the report script	Prose does not change when numbers do	Guarded: generated files hold tables and definitions	Keep interpretation in reviewed documents	D.8
Claims resting on statements	Hardware or audit claims lack an artifact	Guarded: unrecorded facts marked as such	Record facts in run outputs	D.8

D.4 Labels Made by Models

Three of the four tasks have no human labels. Their reference labels come from two language-model judges, `claude-opus-5` from Anthropic and `gpt-5.6-sol` from OpenAI. The judge labels are AI-generated by these two models, not human judgments, and they may be wrong. They are released as evaluation labels so that readers can check this benchmark's numbers; the ESCI human labels are the only human labels in the pack. Language-model judges are now common in recommender evaluation, and Chapter 12 (Section 12.6.4) advises calibrating them against human labels before trusting them. This section shows what changes when that calibration is not possible for most of a benchmark.

D.4.1 What the Agreement Filter Selects

Keeping only the fields on which both judges agree sounds like a quality filter, and in one sense it is: a label two independent judges share is less likely to be an accident. It is also a selection rule, and it does not select at random. On compatibility, which has a single field, the judges agreed on 1,633 of 2,000 items, and the rest were never scored. On query segmentation the judges agreed on all four fields for 68 percent of queries, and Figure D.3 shows how unevenly that agreement falls across the sampling strata: 91 percent for queries that ask for a bundle or set, 48 percent for queries the enrichment marked as ambiguous, and 56.5 percent across the 1,000 ORCAS-I queries. Pooled over the 1,000 queries drawn from the Shopping Queries Dataset, agreement on all four fields is 79.2 percent.

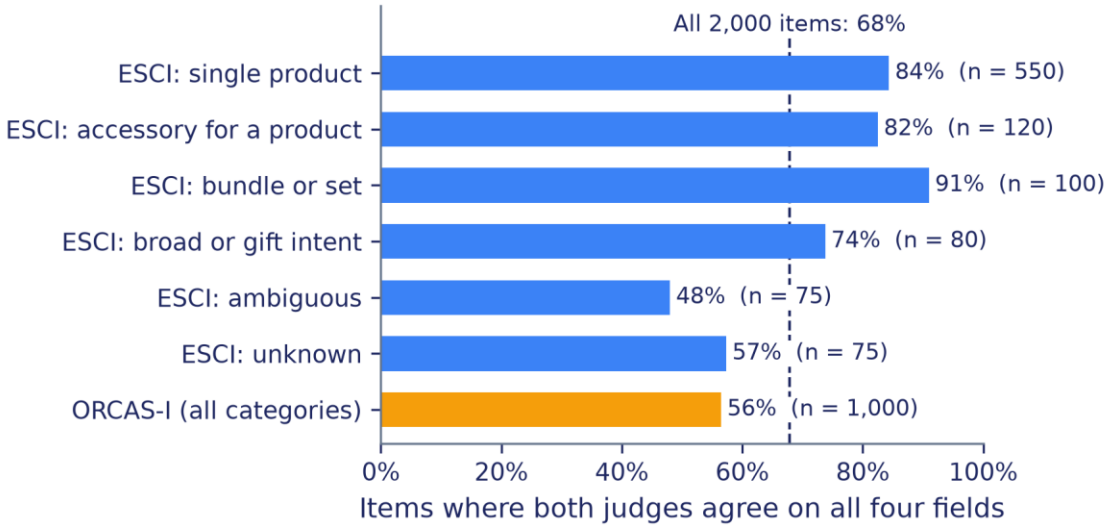


Figure D.3: Share of query-segmentation items on which both judges agree on all four fields, by sampling stratum. The ORCAS-I queries form one merged stratum. The dashed line marks agreement across all 2,000 items. Bar labels are rounded to the nearest percent; the ORCAS-I share in the text is 56.5 percent.

The filter works field by field, so a query can contribute its goal label even when the judges disagree about its specificity. Even so, the scored items lean toward the cases the judges found easy, and the class mix shifts with them. On the query goal field, 857 of the 1,000 ORCAS-I queries kept a label, and on those queries the majority prior reached 6.2 percent accuracy, against 79.1 percent on the largest ESCI stratum, because the prior's class is common among ESCI shopping queries and rare among web queries. Plank (2022) argues that such disagreement is often a property of the item rather than noise to be removed, and the items removed here are the ambiguous queries on which a production system is most likely to go wrong.

D.4.2 How Much the Label Source Matters

A direct test of dependence on the label source scores every scorer three times: against the consensus on agreed items, and against each judge's labels on all items. Two patterns appear. First, absolute accuracy moves. On compatibility, the hosted typed-decision API reaches 74.0 percent on the consensus items but 68.2 and 67.2 percent against each judge on all items, and the majority prior falls from 70.9 percent to 64.5 and 61.4 percent. Second, the order of scorers barely moves. Across the nine judge-labelled fields, only one pair of scorers changes places between label sources: the open adapter mid and the option-letter adapter on query specificity.

A stable order is reassuring but does not establish that the labels are right. Both judges and all five scorers are language models or are derived from them, and models can favour answers shaped like their own: Zheng et al. (2023) documented position, verbosity, and self-enhancement biases in model judges, and Panickssery et al. (2024) linked self-preference to a model's ability to recognise its own outputs. A scorer that shares a judge's priors can look accurate because it shares the judge's mistakes. Northcutt et al. (2021) showed that even modest error rates in test labels can change which model a benchmark selects. The ESCI slice exists to anchor the comparison in human judgments, and human labels carry their own errors, so its scores measure agreement with annotators rather than accuracy against truth.

D.4.3 Degenerate Fields

The multi-target field asks whether a query targets several products. Among the 1,996 items on which the judges agreed, one class holds 1,995. Accuracy on this field measures little beyond how often a scorer says "no": the majority prior scores 99.9 percent, and the encoder decision model scores 1.2 percent because it answers "yes" almost everywhere, a pattern Section D.5 explains. This is the truncated encoder run described in Section D.2. A field like this belongs in a separate note, never in a headline table, and a minimum class support rule decided in advance prevents such a field from being quietly averaged into a summary.

D.5 Probabilities and the Meaning of "Confidence"

A gate or a ranking feature acts on a number attached to a decision. Chapter 9 (Section 9.3.7) shows where the absolute probability enters the money: the score equals the bid times the probability of a click times the probability of a conversion. The number is calibrated when it matches the frequency of the outcome, so that a model which says 0.8 is right about 80 percent of the time, the check Chapter 15 (Section 15.5.2) uses for monitoring. For a closed question with k options, the number a decision uses is the top-label probability p, the probability the scorer assigned to the option it returned. The expected calibration error (Guo et al., 2017) sorts the n scored items into M bins by p and compares, within each bin B_m, the accuracy $\mathrm{acc}(B_m)$ with the mean probability $\bar{p}(B_m)$:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\mathrm{acc}(B_m) - \bar{p}(B_m)\right|$$

This appendix reports top-label ECE with 15 bins of equal width and, as a check, 15 bins of equal size.

D.5.1 Three Meanings of One Field

Every scorer also returns a field called `confidence`, which sounds like the probability of being right. The replay report tests which formula reproduces each reported value to within 0.01, and three different formulas do. On every labelled item of the four headline fields, the two open adapters report the rescaled margin, a formula the hosted API also documents (TypeSafe, 2026a):

$$c_{\mathrm{margin}} = \frac{p - 1/k}{1 - 1/k}$$

which places a uniform answer at 0 and certainty at 1. The option-letter adapter reports p itself, as its card states (Vagmi, 2026). The encoder decision model reports one minus the normalised entropy of the probability vector $(p_1, \ldots, p_k)$:

$$c_{\mathrm{entropy}} = 1 + \frac{\sum_{j=1}^{k} p_j \log p_j}{\log k}$$

For the hosted API, the documented formula matches on 75.8 to 79.5 percent of labelled items across the four fields, and the committed files do not explain the remainder. Figure D.4 plots reported `confidence` against p on the compatibility task.

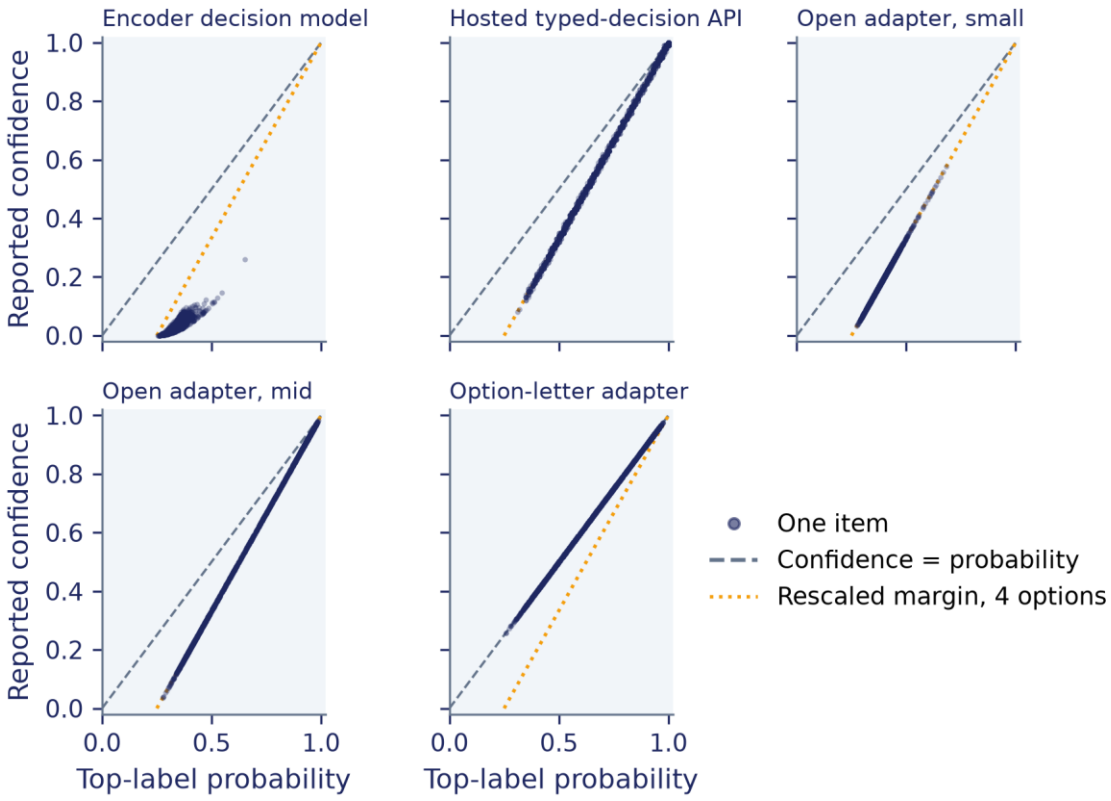


Figure D.4: Reported `confidence` against top-label probability on the compatibility task, one point per item. The open adapters and most hosted-API answers follow the rescaled margin, the option-letter adapter follows the diagonal, and the encoder decision model follows neither. The encoder decision model ran with the adapter's `max_len=512`, although the checkpoint accepts 1,024 tokens; a tokenizer check on 2026-11-09 found inputs cut at 512 tokens for 3 of the 2,000 compatibility items.

The differences are large: on compatibility, the encoder decision model's mean top-label probability is 0.337 and its mean reported `confidence` is 0.031. The hosted API's documentation states that its `confidence` measures how concentrated the probabilities are, not the probability that the selected answer is correct (TypeSafe, 2026a). A field named `confidence` cannot be assumed to share a meaning across vendors or versions until it has been checked against the probability vector.

D.5.2 Two Calibration Numbers

Binning `confidence` as though it were p produces a number that looks like calibration error and is not. Figure D.5 sets the two side by side. On compatibility, the open adapter small has an ECE on reported `confidence` of 0.053 and a top-label ECE of 0.243: the rescaled margin sits below p by construction, and here that offset happens to land near the scorer's accuracy. On query goal, the encoder decision model moves the other way, from 0.105 on p to 0.348 on its entropy-based `confidence`. For the option-letter adapter the two coincide.

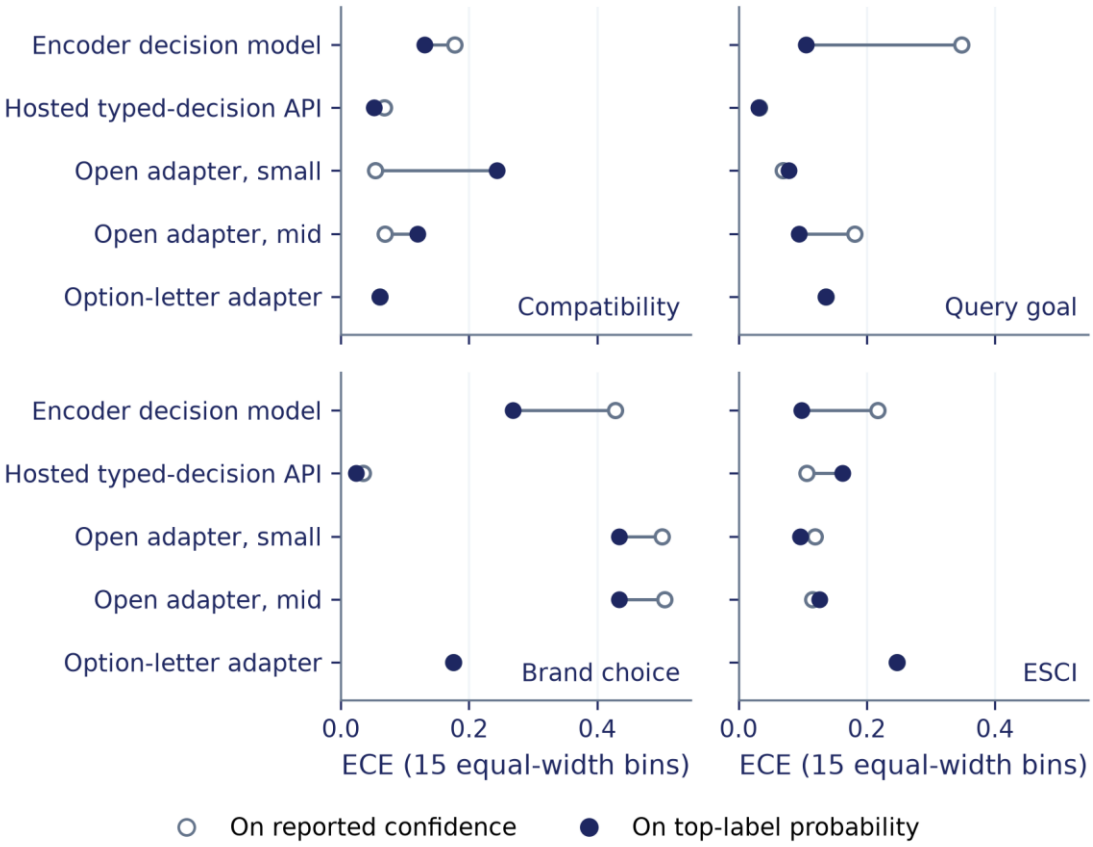


Figure D.5: Expected calibration error computed on reported `confidence` (open circles) and on top-label probability (filled circles) for four fields. Only the filled circles measure calibration. The encoder decision model ran with the adapter's `max_len=512`, although the checkpoint accepts 1,024 tokens. A tokenizer check on 2026-11-09 found inputs cut at 512 tokens for 1,996 of the 2,000 brand and category items and 3 of the 2,000 compatibility items; the checkpoint's 256-token question limit kept 127 to 215 of the 452 instruction tokens in each query-segmentation question; ESCI inputs were not cut.

Top-label ECE has limits of its own. Binned estimators are biased and sensitive to the binning (Nixon et al., 2019; Kumar et al., 2019): on query object, the encoder decision model scores 0.063 with equal-width bins and 0.091 with equal-mass bins. The majority prior assigns probability one to a single class, so its ECE equals its error rate, 0.291 on compatibility, and any scorer that predicts one class with matching probability can score well while separating nothing. ECE also hides direction. On brand choice, both open adapters have top-label ECE near 0.43 because their mean top-label probabilities sit below 0.47 while their accuracies exceed 85 percent. They are underconfident, and a gate that read their probabilities at face value would hold back answers that are usually right.

D.5.3 Proper Scoring Rules and Ranking

Proper scoring rules reward a scorer for reporting its true beliefs and use the whole probability vector (Gneiting and Raftery, 2007). Log loss is the mean negative log probability of the reference label; the multiclass Brier score is the squared distance between the probability vector and the one-hot reference, averaged over items (Brier, 1950). They can disagree sharply. On ESCI, the hosted API placed probability zero on the reference label for 236 of 8,000 pairs. With probabilities floored at 10^{-12}, those 236 items

account for a large share of its log loss of 1.786, the largest of the five scorers on that field, while its Brier score of 0.566 is the smallest, because Brier bounds the penalty for a confident miss.

A gate needs to know whether the probability ranks right answers above wrong ones. The area under the risk-coverage curve (AURC) ranks items by p and accumulates the error rate as coverage grows (Geifman and El-Yaniv, 2017). The underconfident open adapter small reaches an AURC of 0.042 on brand choice, against 0.415 for the majority prior: its probabilities rank answers well even though their scale is wrong, so a threshold chosen on one's own labelled data could still use them.

Two smaller choices complete the decision rule. A yes-or-no threshold of 0.5 is a default rather than a decision: on the multi-target field, the encoder decision model is right 1.2 percent of the time, and the probability of the label it returns averages 0.565, so a cut at 0.5 turns a score hovering near one half into a "yes" for almost every query. This is the truncated encoder run described in Section D.2. The hosted API's documentation suggests starting with a middle band that gets its own outcome (TypeSafe, 2026a). Ties need a rule too: the hosted API occasionally returned a label whose probability was below the maximum of its own vector, on 5 of 8,000 ESCI pairs and on 14 of 26,000 item-field pairs across all ten fields, and the option-letter adapter's ESCI vectors tied at the maximum on 295 of 8,000 pairs. A benchmark should state which label it scores and how ties are broken.

D.6 Order, Wording, and Other Inputs

A typed decision depends on details a human reader would consider irrelevant: the order of the options, the wording of the question, how much of the input survives truncation, and whether the model saw the test items during training.

D.6.1 Option Order

Language models answering multiple-choice questions favour some option positions or identifiers over others. Zheng et al. (2024) traced much of this to a preference for particular option tokens, such as the letter "A", and proposed estimating that prior by permuting the options. Pezeshkpour and Hruschka (2024) found order sensitivity strongest when a model is uncertain between its top candidates. Unlike the position bias of Chapter 3 (Section 3.3.4), where users click higher-ranked items more often, here the model prefers a position, and whoever assembled the option list chose it.

The benchmark shuffles the brand and category candidates with a seeded generator, so the reference option appears at each of the eight positions on 100 to 147 labelled items, and the replay report breaks accuracy down by that position. The hosted API is nearly flat, with brand-choice accuracy between 88.0 and 99.0 percent across positions. The encoder decision model is right on 64.0 percent of brand-choice items when the reference brand is listed first and on 14.0 percent when it is last, while the open adapter mid shows the reverse, from 28.0 percent first to 89.0 percent last. On category choice, the encoder decision model reaches 37.9 percent when the reference category is fifth but 1.6 percent when it is first and 1.4 percent when it is last. The encoder rows are the truncated run in Section D.2, which cut 1,996 of these 2,000 brand and category items at 512 tokens.

Shuffling spreads such biases evenly, so the averages in Table D.4 are fair to every scorer, but it does nothing for a single decision. Production option lists are rarely shuffled: brands might be sorted by popularity, and a scorer that prefers late positions would then make the same mistake on every query. Two guards help: report accuracy by position, remembering that a little over a hundred items per position makes differences of a few points noise, and score each item under several orders and average the probabilities, which the open adapters' server supports with a permutation endpoint (Palmer, 2026).

D.6.2 Wording and Format

Sclar et al. (2024) showed that formatting changes that leave the meaning intact, such as separators, spacing, and casing, can move accuracy by large margins and change which of two models looks better. Zhao et al. (2021) identified biases toward labels that are frequent or recent in the prompt or common in pre-training text, and Thomas et al. (2024) saw label quality move with simple paraphrases. The benchmark used one prompt version for every scorer, so each of its numbers is conditional on one wording. Scoring several reviewed paraphrases and reporting the spread, as Exercise D.2 describes, shows how much of each difference survives a change of wording, and Chapter 12 (Section 12.8.2) describes how to version prompts so that such comparisons stay reproducible.

D.6.3 Truncation, Contamination, and Setting

Truncation removes evidence silently. For the encoder decision model, Section D.2 records both cuts: 1,996 of the 2,000 brand and category items were cut at 512 tokens, and each query-segmentation question kept 127 to 215 of its 452 instruction tokens, so the model answered with part of its instructions missing. The guard is cheap: count tokens with each model's own tokenizer and report the share of inputs cut.

Contamination cannot be ruled out. The ESCI labels have been public since 2022 (Reddy et al., 2022), and the hosted model's training data is not disclosed. Chapter 12 (Section 12.6.2) treats leakage of this kind; the safe reading of the ESCI rows is that they may be optimistic for data no model has seen. Finally, every run was zero-shot. The encoder and open-adapter projects both report that fine-tuning on one's own decisions raises accuracy and refits the probabilities (Convai Innovations, 2026; Palmer, 2026), and none of that is measured here.

D.7 Reading a Results Table Without Picking a Winner

Table D.4 gives the benchmark's results on four fields. It is dated because the scorers are products that change, and it lists the majority prior first and the scorers in a fixed order that carries no ranking.

Table D.4: Results on four fields from the September 2026 runs, recomputed from the replay pack built on 2026-11-09. Intervals are 95 percent paired bootstrap intervals. The encoder decision model ran with the adapter's `max_len=512`, although the checkpoint accepts 1,024 tokens. A tokenizer check on 2026-11-09 found inputs cut at 512 tokens for 1,996 of the 2,000 brand and category items and 3 of the 2,000 compatibility items; the checkpoint's 256-token question limit kept 127 to 215 of the 452 instruction tokens in each query-segmentation question; ESCI inputs were not cut.

Field	Scorer	n	Accuracy, % [95% interval]	Macro-F1, fixed [95% interval]	Top-label ECE [95% interval]	Log loss
Compatibility	Majority prior	1,633	70.9 [68.6, 73.1]	0.207 [0.203, 0.211]	0.291 [0.269, 0.314]	8.0

Field	Scorer	n	Accuracy, % [95% interval]	Macro-F1, fixed [95% interval]	Top-label ECE [95% interval]	Logloss
						37
Compatibility	Encoder decision model	1,633	20.8 [18.8, 22.8]	0.151 [0.132, 0.171]	0.130 [0.111, 0.151]	1.598
Compatibility	Hosted typed-decision API	1,633	74.0 [71.8, 76.1]	0.569 [0.524, 0.609]	0.052 [0.038, 0.068]	0.614
Compatibility	Open adapter, small	1,633	15.8 [14.1, 17.6]	0.133 [0.112, 0.156]	0.243 [0.225, 0.260]	1.411
Compatibility	Open adapter, mid	1,633	59.8 [57.4, 62.1]	0.295 [0.269, 0.325]	0.120 [0.102, 0.141]	0.943
Compatibility	Option-letter adapter	1,633	76.3 [74.3, 78.3]	0.428 [0.389, 0.464]	0.061 [0.053, 0.083]	0.656
Query goal	Majority prior	1,788	42.4 [40.2, 44.7]	0.099 [0.096, 0.103]	0.576 [0.553, 0.598]	15.902

Field	Scorer	n	Accuracy, % [95% interval]	Macro-F1, fixed [95% interval]	Top-label ECE [95% interval]	Logloss
Query goal	Encoder decision model	1,788	47.9 [45.6, 50.2]	0.328 [0.301, 0.357]	0.105 [0.085, 0.127]	1.362
Query goal	Hosted typed-decision API	1,788	79.3 [77.3, 81.2]	0.762 [0.728, 0.789]	0.032 [0.025, 0.053]	0.512
Query goal	Open adapter, small	1,788	27.6 [25.6, 29.7]	0.306 [0.255, 0.347]	0.078 [0.060, 0.099]	1.584
Query goal	Open adapter, mid	1,788	60.1 [57.8, 62.4]	0.536 [0.495, 0.574]	0.095 [0.083, 0.116]	1.070
Query goal	Option-letter adapter	1,788	78.6 [76.7, 80.4]	0.712 [0.674, 0.746]	0.136 [0.124, 0.155]	0.702
Brand choice	Majority prior	1,947	56.3 [54.1, 58.5]	0.080 [0.078, 0.082]	0.437 [0.415, 0.459]	12.077
Brand choice	Encoder decision model	1,947	45.4 [43.1, 47.7]	0.393 [0.367, 0.418]	0.268 [0.247, 0.292]	1.9

Field	Scorer	n	Accuracy, % [95% interval]	Macro-F1, fixed [95% interval]	Top-label ECE [95% interval]	Log loss
						79
Brand choice	Hosted typed-decision API	1,947	96.4 [95.5, 97.2]	0.948 [0.935, 0.960]	0.024 [0.020, 0.032]	0.129
Brand choice	Open adapter, small	1,947	89.8 [88.5, 91.2]	0.851 [0.831, 0.872]	0.434 [0.422, 0.447]	0.873
Brand choice	Open adapter, mid	1,947	85.6 [84.1, 87.2]	0.775 [0.750, 0.798]	0.435 [0.420, 0.449]	0.984
Brand choice	Option-letter adapter	1,947	92.9 [91.7, 94.0]	0.911 [0.895, 0.925]	0.175 [0.164, 0.187]	0.392
ESCI, human labels	Majority prior	8,000	25.0 [24.1, 26.0]	0.100 [0.097, 0.103]	0.750 [0.740, 0.759]	20.723
ESCI, human labels	Encoder decision model	8,000	25.5 [24.5, 26.5]	0.184 [0.176, 0.192]	0.098 [0.089, 0.108]	1.409

Field	Scorer	n	Accuracy, % [95% interval]	Macro-F1, fixed [95% interval]	Top-label ECE [95% interval]	Log loss
ESCI, human labels	Hosted typed-decision API	8,000	60.4 [59.3, 61.4]	0.603 [0.592, 0.614]	0.162 [0.152, 0.172]	1.786
ESCI, human labels	Open adapter, small	8,000	30.9 [29.9, 32.0]	0.226 [0.217, 0.235]	0.096 [0.086, 0.106]	1.406
ESCI, human labels	Open adapter, mid	8,000	45.8 [44.7, 46.9]	0.414 [0.404, 0.425]	0.126 [0.115, 0.137]	1.281
ESCI, human labels	Option-letter adapter	8,000	43.2 [42.2, 44.3]	0.395 [0.384, 0.407]	0.247 [0.236, 0.258]	1.452

The first three fields are scored against judge consensus labels and the fourth against human labels. Top-label ECE uses 15 equal-width bins on the probability of the returned label, not on the reported `confidence`. Because of the truncation stated in the title, the encoder decision model's rows measure this configuration of an out-of-domain specialist checkpoint, not the checkpoint's best case.

D.7.1 Baselines and Class Balance

Read the baseline row first. On compatibility the majority prior reaches 70.9 percent accuracy, because most candidate pairs in the agreed set are incompatible: 1,158 of the 1,633 agreed items carry that class. Three of the five scorers fall below it. Accuracy on an imbalanced field rewards guessing the common class, the same trap that Chapter 3 (Section 3.6.3) guards against with random and popularity baselines for rankers. Macro-F1 averages per-class F1, so the prior's 0.207 on compatibility exposes how little it knows. Macro-F1 has more than one definition in use (Opitz and Burst, 2019); Table D.4 averages over every class in the field's label space, so a class that a scorer never predicts counts as zero rather than disappearing from the average. ESCI is balanced at 2,000 pairs per grade, so its prior sits at 25.0

percent, and calibration measured on a balanced slice need not carry over to the natural ESCI label mix, in which Exact pairs dominate (Reddy et al., 2022).

D.7.2 Intervals and Paired Comparisons

Each interval comes from a bootstrap over items with 2,000 resamples and a fixed seed, and every scorer on a field is resampled with the same item indices. Pairing matters because item difficulty is shared: an easy resample lifts every scorer, so the difference between two scorers is far less noisy than either score (Berg-Kirkpatrick et al., 2012; Dror et al., 2018). Chapter 16 (Section 16.4.1) reviews the statistics, and Card et al. (2020) showed that many evaluation sets are too small to detect the modest differences that papers report.

The paired differences show why no single ordering emerges. On compatibility, the hosted API minus the option-letter adapter is −2.3 accuracy points with an interval from −4.7 to +0.1, which includes zero, while the same pair's macro-F1 difference of +0.141 has an interval from +0.096 to +0.188, which does not. Model size does not settle the order either: on brand choice the open adapter small exceeds the open adapter mid by 4.2 points (interval +2.6 to +5.8), and on compatibility it trails by 44.0 points (interval −46.9 to −41.0). Averages also hide strata: on the 333 labelled hard-negative substitute pairs, the hosted API's compatibility accuracy is 36.0 percent and the option-letter adapter's is 66.4 percent, while both exceed 99 percent on products from other categories.

The replay report runs 135 paired comparisons per metric, and the interval excludes zero in 127 of them for accuracy, 129 for macro-F1, and 118 for top-label ECE. With that many comparisons, some exclusions would appear by chance even if no real differences existed, a problem Chapter 16 (Section 16.4.6) addresses. The defence is to name the few comparisons that matter for a decision before reading the table.

D.7.3 What the Table Cannot Support

The table supports statements of the form "on this field, with these labels, this scorer's accuracy exceeds that one's by an amount in this interval". It does not support a winner. It also cannot support any latency comparison: the runs used different hardware, batch sizes of 8 and 32, and, for the hosted API, a network path, and the committed runs recorded neither GPU types nor checkpoint revisions. Chapter 14 (Section 14.3) sets out the conditions a latency comparison needs. Finally, the table describes one prompt version and zero-shot use. The query-goal field mixes English shopping queries with a web-search stratum; the other three fields are English commerce query-product pairs. The product versions have since moved on.

D.8 Reproducing the Scores on a CPU

Every benchmark number in this appendix can be recomputed without a GPU, an API key, or the licensed text. The companion code ships a replay pack of 28 compressed JSON Lines files: the reference labels for each task and the predictions of each of the six rows on each task, with item identifiers, returned labels, full probability vectors, reported `confidence`, and recorded time per item. A manifest records the SHA-256 hash of every file, the repository commit, the prompt version, each scorer's role label, the rubric hashes, the source-data licences, and the build checks. A second file records judge agreement by stratum and when each set of labels was generated.

The pack holds no query or product text: its builder refuses to write it if any string falls outside a closed vocabulary of task, field, label, stratum, and label-source names, or if any ORCAS-I intent category name

appears. The cost is that prompts, model inputs, and judge rationales cannot be inspected from the pack.

Three commands, run from the benchmark directory of the companion code, cover the checks:

```
python replay_scores.py verify
python replay_scores.py report --output-dir /tmp/replay
python replay_scores.py figures --output-dir /tmp/replay
```

The `verify` command checks every file against the manifest hashes, then recomputes every value stored in each run's summary file at the stored precision: 4,092 values, of which 3,852 are compared one by one and 240 in aggregate, because the summaries break ORCAS-I accuracy out by intent category and the pack does not carry that category. It exits with 0 when everything matches, 1 when a metric or the results file differs, and 2 when the pack does not match its manifest. The `report` command regenerates the replay results and JSON metrics behind every interval, calibration estimate, and breakdown quoted here, and the `figures` command redraws Figures D.3 to D.5. Writing both to a temporary directory leaves the committed files untouched. The regenerated report and metrics match the files committed in the repository byte for byte. A Windows checkout that rewrites line endings will differ from that output only by those endings; `verify` still exits 0, because it compares the text after normalising newlines. On Windows, `--output-dir /tmp/replay` writes under `C:\tmp\replay`.

Three guards from Table D.3 belong to this layer. The prediction cache is keyed on a hash of the scorer, model, task, item, and a hand-set prompt version string, so editing a question without changing that string would return stale predictions. Keying on a hash of the rendered prompt closes that gap. Chapter 12 (Section 12.8.2) is the production form of the same discipline: keep prompts in version control, and record the prompt version and the model identifier with every output. The report script writes tables and definitions only, so interpretation lives in reviewed text and cannot drift from the numbers. Facts a run did not record, such as GPU types, are marked as not recorded rather than filled in from memory.

D.9 Using a Structured-Output Scorer in Production

A public benchmark can narrow a shortlist; it cannot choose a scorer for a particular catalog. The methods in this appendix translate into a short sequence of checks on a team's own data before a typed decision reaches a filter, a feature, or a gate.

Start with a gold set drawn from your own traffic: a few hundred items labelled by people who know the catalog, stratified so that hard negatives and ambiguous queries are well represented rather than filtered away. The Bing study kept such a set for exactly this purpose, to choose prompts and to audit model labels (Thomas et al., 2024). If language-model judges label the larger evaluation set, measure their agreement with the gold set first, as Chapter 12 (Section 12.6.4) advises, and avoid judging a scorer with a model from its own family (Panickssery et al., 2024).

Next, establish what each number means. Recompute the scorer's `confidence` from its probability vector to learn which statistic it is, then place thresholds on the probability of the option the code will act on, as the hosted API's own documentation advises (TypeSafe, 2026a). Choose each threshold on held-out labelled data and read off the error rate and coverage at that point, using a stricter threshold for actions that are costly to undo. In a cascade, the same thresholds decide which items escalate to a slower model or a person (Chapter 12, Section 12.4.4).

Then test the inputs that the benchmark could not. Score the gold set with the option lists in the order production will use, and again under several permutations and reviewed paraphrases of each question;

Exercise D.2 sketches this check on a single GPU. Count tokens with the scorer's own tokenizer so that truncation is measured rather than discovered.

Finally, pin and record. Send a versioned model identifier rather than an alias: the hosted API's documentation notes that an alias moves when a new release ships, so its answers can change with no change to the calling code, and it recommends sending the version identifier when thresholds have been tuned against one version (TypeSafe, 2026c). For open checkpoints, record the revision hash. Log every decision with the model version, a hash of the rendered prompt, a hash of the input, and the full probability vector. Chapter 15 (Section 15.9.3) requires the rest of the serving system to be named and retrievable as well: the training data and its window, the feature definitions, the weights, the embedding tables, the nearest-neighbour index, and the decision policy. Monitor the distribution of answers and of top-label probabilities over time, alongside the calibration checks of Chapter 15 (Section 15.5.2), and rerun the gold set before moving any threshold to a new version.

Summary

Typed decision models return closed answers with a probability for every option, which removes parsing failures and moves the hard questions elsewhere: whether the answer is right, whether its probability means what the consumer assumes, and whether it survives changes that should not matter. A benchmark that answers those questions needs traceable items and labels, a baseline row, and scores that anyone can recompute. The benchmark in this appendix meets that bar with a text-free replay pack that reproduces every number on a CPU.

Its lessons carry beyond its dated results. Labels from language-model judges are evidence, not truth: an agreement filter selects easier items, and only a human-labelled anchor checks circularity. A field named `confidence` meant three different things across five scorers, so calibration has to be computed from the probability a decision uses. Proper scoring rules and risk-coverage curves answer different questions from ECE, and a scorer can be poorly calibrated yet still rank its answers well. Option order changed some scorers' answers by tens of points, and one prompt wording leaves every result conditional. Paired intervals and named comparisons keep a results table from turning into a leaderboard, and a team's own gold set, pinned versions, and logged probability vectors carry the same discipline into production.

Exercises

Exercise D.1: Recomputing Calibration From the Replay Pack

This exercise needs only a CPU. Run the `verify` command from Section D.8 and confirm that it exits with 0.

Part A. Load the compatibility labels and two scorers' predictions from the pack. Compute top-label ECE from the probability vectors and ECE on the reported `confidence`, each with 15 equal-width bins, and check both against the replay report. Repeat with 5, 10, 20, and 30 bins and with equal-mass bins. Look at how far each estimate moves, and whether the gap between the two calibration numbers survives every choice.

Part B. For brand choice, sort the agreed items by top-label probability and trace the error rate as coverage grows. Pick the threshold that keeps the error rate below 5 percent on the consensus labels, then apply it to all items and score it against each judge's labels. Note how much coverage and error rate change with the label source.

Exercise D.2: Order and Wording on One GPU (Optional)

This exercise needs a single GPU, such as an L4 on a Colab runtime, which the open adapters' documentation lists as sufficient for the small model (Palmer, 2026). It uses only public data, and this appendix reports no results for it.

Part A. Serve the open adapter small with its project's server, and build a small balanced slice of ESCI pairs from the public Shopping Queries Dataset with the companion code's slice builder, passing a small `--human-per-label` (the builder's default, 2,000, is the slice this appendix already reports). Score each pair under all 24 orders of the four grades by calling the server's permutation endpoint with `n_perm` set to 24 (the documented default is 6). Compare accuracy and top-label ECE under the original order with those after averaging probabilities across orders, and count the pairs whose answer changes under at least one order.

Part B. Write three paraphrases of the ESCI question and its grade descriptions, and have someone check that each preserves the meaning. Score the same pairs under each wording with paired bootstrap intervals for the differences. Look for whether the spread across wordings exceeds the intervals in Table D.4.

References

Alexander, D., Kusa, W., and de Vries, A. P. (2022). "ORCAS-I: Queries Annotated with Intent using Weak Supervision." Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval. ACM. https://doi.org/10.1145/3477495.3531737

Berg-Kirkpatrick, T., Burkett, D., and Klein, D. (2012). "An Empirical Investigation of Statistical Significance in NLP." Proceedings of the 2012 Joint Conference on Empirical Methods in Natural Language Processing and Computational Natural Language Learning. Association for Computational Linguistics. https://aclanthology.org/D12-1091/

Brier, G. W. (1950). "Verification of Forecasts Expressed in Terms of Probability." Monthly Weather Review 78(1). American Meteorological Society. https://journals.ametsoc.org/view/journals/mwre/78/1/1520-0493_1950_078_0001_vofeit_2_0_co_2.xml

Card, D., Henderson, P., Khandelwal, U., Jia, R., Mahowald, K., and Jurafsky, D. (2020). "With Little Power Comes Great Responsibility." Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics. https://aclanthology.org/2020.emnlp-main.745/

Convai Innovations. (2026). "Laya." Hugging Face Model Card. https://huggingface.co/convaiinnovations/laya

Dror, R., Baumer, G., Shlomov, S., and Reichart, R. (2018). "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing." Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics. Association for Computational Linguistics. https://aclanthology.org/P18-1128/

Geifman, Y., and El-Yaniv, R. (2017). "Selective Classification for Deep Neural Networks." Advances in Neural Information Processing Systems 30. Curran Associates. https://arxiv.org/abs/1705.08500

Gneiting, T., and Raftery, A. E. (2007). "Strictly Proper Scoring Rules, Prediction, and Estimation." Journal of the American Statistical Association 102(477). Taylor and Francis. https://doi.org/10.1198/016214506000001437

Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). "On Calibration of Modern Neural Networks." Proceedings of the 34th International Conference on Machine Learning. PMLR. https://arxiv.org/abs/1706.04599

Kumar, A., Liang, P., and Ma, T. (2019). "Verified Uncertainty Calibration." Advances in Neural Information Processing Systems 32. Curran Associates. https://arxiv.org/abs/1909.10155

Nixon, J., Dusenberry, M. W., Zhang, L., Jerfel, G., and Tran, D. (2019). "Measuring Calibration in Deep Learning." Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops. IEEE. https://arxiv.org/abs/1904.01685

Northcutt, C. G., Athalye, A., and Mueller, J. (2021). "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks." Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks. https://arxiv.org/abs/2103.14749

Opitz, J., and Burst, S. (2019). "Macro F1 and Macro F1." arXiv preprint arXiv:1911.03347. https://arxiv.org/abs/1911.03347

Palmer, J. (2026). "Kev." GitHub Repository. https://github.com/jaredpalmer/kev

Panickssery, A., Bowman, S. R., and Feng, S. (2024). "LLM Evaluators Recognize and Favor Their Own Generations." Advances in Neural Information Processing Systems 37. Curran Associates. https://arxiv.org/abs/2404.13076

Pezeshkpour, P., and Hruschka, E. (2024). "Large Language Models Sensitivity to The Order of Options in Multiple-Choice Questions." Findings of the Association for Computational Linguistics: NAACL 2024. Association for Computational Linguistics. https://arxiv.org/abs/2308.11483

Plank, B. (2022). "The 'Problem' of Human Label Variation: On Ground Truth in Data, Modeling and Evaluation." Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics. https://doi.org/10.18653/v1/2022.emnlp-main.731

Reddy, C. K., Màrquez, L., Valero, F., Rao, N., Zaragoza, H., Bandyopadhyay, S., Biswas, A., Xing, A., and Subbian, K. (2022). "Shopping Queries Dataset: A Large-Scale ESCI Benchmark for Improving Product Search." arXiv preprint arXiv:2206.06588. https://arxiv.org/abs/2206.06588

Sclar, M., Choi, Y., Tsvetkov, Y., and Suhr, A. (2024). "Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design or: How I Learned to Start Worrying about Prompt Formatting." Proceedings of the Twelfth International Conference on Learning Representations. https://arxiv.org/abs/2310.11324

Tam, Z. R., Wu, C.-K., Tsai, Y.-L., Lin, C.-Y., Lee, H.-y., and Chen, Y.-N. (2024). "Let Me Speak Freely? A Study on the Impact of Format Restrictions on Large Language Model Performance." Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track. Association for Computational Linguistics. https://aclanthology.org/2024.emnlp-industry.91/

Thomas, P., Spielman, S., Craswell, N., and Mitra, B. (2024). "Large Language Models Can Accurately Predict Searcher Preferences." Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval. ACM. https://arxiv.org/abs/2309.10621

TypeSafe. (2026a). "Confidence." TypeSafe Documentation. https://docs.typesafe.ai/confidence

TypeSafe. (2026b). "Choice." TypeSafe Documentation. https://docs.typesafe.ai/primitives/choice

TypeSafe. (2026c). "Models." TypeSafe Documentation. https://docs.typesafe.ai/models

Vagmi. (2026). "jev-lite." Hugging Face Model Card. https://huggingface.co/vagmi/jev-lite

Willard, B. T., and Louf, R. (2023). "Efficient Guided Generation for Large Language Models." arXiv preprint arXiv:2307.09702. https://arxiv.org/abs/2307.09702

Zhao, Z., Wallace, E., Feng, S., Klein, D., and Singh, S. (2021). "Calibrate Before Use: Improving Few-Shot Performance of Language Models." Proceedings of the 38th International Conference on Machine Learning. PMLR. https://arxiv.org/abs/2102.09690

Zheng, C., Zhou, H., Meng, F., Zhou, J., and Huang, M. (2024). "Large Language Models Are Not Robust Multiple Choice Selectors." Proceedings of the Twelfth International Conference on Learning Representations. https://arxiv.org/abs/2309.03882

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., and Stoica, I. (2023). "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena." Advances

in Neural Information Processing Systems 36, Datasets and Benchmarks Track. Curran Associates. https://arxiv.org/abs/2306.05685